Министерство науки и высшего образования Российской Федерации
Федеральное государственное бюджетное образовательное учреждение высшего образования
РХТУ им. Д.И. Менделеева

ЛАБОРАТОРНАЯ РАБОТА № 1

«Расчёт собственных чисел и собственных векторов
симметричной матрицы методом Якоби»

Выполнил: Ф.И.О., группа — Елфимова Карина Владимировна, КС-43
Проверил: Ф.И.О. — Сукин Иван Андреевич

Москва, 2026

СОДЕРЖАНИЕ

Оглавление

1. ЦЕЛЬ РАБОТЫ 2

2. ТЕОРЕТИЧЕСКИЕ СВЕДЕНИЯ 3

2.1. Постановка задачи 3

2.2. Идея метода Якоби 3

2.3. Шаг поворота 4

2.4. Норма внедиагональной части и критерий остановки 4

2.5. Выбор пары и сложность 5

3. ОПИСАНИЕ ПРОГРАММНОГО ПАКЕТА 6

4. ХОД РАБОТЫ 7

4.1. Ядро метода (src/jacobi.rs) 7

4.2. Функция входа в решение (src/solver.rs) 7

4.3. Консольный ввод/вывод (src/console_io.rs) 8

4.4. Документация и тесты 8

ЗАКЛЮЧЕНИЕ 10

ПРИЛОЖЕНИЕ А 11

ПРИЛОЖЕНИЕ Б 27

ИСТОЧНИКИ 27

1. ЦЕЛЬ РАБОТЫ

Реализовать на языке Rust численный метод Якоби вычисления собственных чисел и собственных векторов действительной симметричной матрицы, оформить разработку как пакет (crate) с публичным интерфейсом, консольным приложением, документацией и тестами.

2. ТЕОРЕТИЧЕСКИЕ СВЕДЕНИЯ

2.1. Постановка задачи

Пусть A — действительная симметричная матрица размера n×n. Требуется найти такие λ и ненулевые векторы x, что $Ax = \lambda x$. Для симметричной матрицы все собственные числа вещественны, а собственные векторы, отвечающие разным собственным числам, ортогональны. Следовательно, существует спектральное разложение $A = V \cdot \mathrm{diag}(\lambda_1,\ldots,\lambda_n) \cdot V^T$, где V — ортогональная матрица, столбцы которой — собственные векторы.

2.2. Идея метода Якоби

Метод Якоби основан на последовательном занулении внедиагональных элементов с помощью ортогональных преобразований — поворотов Гивенса. На каждом шаге выбирается пара индексов (p, q) и строится ортогональная матрица P, отличающаяся от единичной только четырьмя элементами (p,p), (p,q), (q,p), (q,q). Преобразование $A \leftarrow P^T \cdot A \cdot P$ сохраняет симметричность и ортогональное подобие, при этом элемент A_pq обнуляется. После конечного числа шагов матрица становится диагональной, а произведение всех поворотов даёт матрицу собственных векторов.

2.3. Шаг поворота

Рассмотрим 2×2 симметричный блок матрицы A, отвечающий индексам p и q:

$$A_2 = \begin{bmatrix} a_{pp} & a_{pq} \\ a_{pq} & a_{qq} \end{bmatrix} \quad (1)$$

Поворот параметруется углом φ. Элемент A_pq после преобразования равен

$$A'_{pq} = a_{pq}(\cos\varphi^2 - \sin\varphi^2) + (a_{pp} - a_{qq}) \sin\varphi \cos\varphi \quad (2)$$

Приравняв выражение к нулю и подставив tgφ = sinφ/cosφ, получаем уравнение на тангенс:

$$\operatorname{tg} 2\varphi = \frac{2\, a_{pq}}{a_{pp} - a_{qq}} \quad (3)$$

Введя обозначение

$$c = \frac{a_{qq} - a_{pp}}{2\, a_{pq}} \quad (4)$$

приводим условие обнуления к квадратному уравнению относительно t = tgφ:

$$t^2 + 2c \cdot t - 1 = 0 \quad (5)$$

Его корни:

$$t = -c \pm \sqrt{c^2 + 1} \quad (6)$$

Выбираем корень с наименьшим по модулю тангенсом (это обеспечивает наилучшую сходимость и исключает деление на ноль):

$$t = \frac{\operatorname{sign}(c)}{|c| + \sqrt{c^2+1}} \quad (7)$$

Если c = 0, берётся t = 1. Отсюда однозначно определяются sinφ и cosφ:

$$\cos\varphi = \frac{1}{\sqrt{1+t^2}}, \quad \sin\varphi = t \cdot \cos\varphi \quad (8)$$

Диагональные элементы обновляются по формулам

$$a_{pp}' = a_{pp} - a_{pq} \cdot t \quad (9)$$

$$a_{qq}' = a_{qq} + a_{pq} \cdot t \quad (10)$$

а остальные элементы строки p, q и столбцы p, q пересчитываются по правилам поворота с сохранением симметрии (a_ij = a_ji).

2.4. Норма внедиагональной части и критерий остановки

В качестве критерия сходимости используется сумма квадратов внедиагональных элементов

$$S = \sum_{i \neq j,} a_{ij}{}^2 \quad (11)$$

Один поворот гарантирует убывание S ровно на $2 \cdot a_pq^2$:

$$S_{new} = S_{old} - 2a_{pq}{}^2 \quad (12)$$

Поскольку S монотонно убывает и ограничена снизу нулём, метод обязан сойтись. Итерации прекращаются, когда $S < \varepsilon$, где ε — заданная точность. Для защиты от бесконечного цикла при неблагоприятных значениях вводится также ограничение на максимальное число итераций.

2.5. Выбор пары и сложность

В реализации применяется лексикографический обход пар (p, q): (1,2), (1,3), …, (n−1,n), затем цикл повторяется. Такой обход прост в реализации и не требует отдельного поиска максимального внедиагонального элемента. Асимптотическая сложность метода Якоби — $O(n^3)$ на полную диагонализацию, что приемлемо для матриц умеренного размера и характеризуется высокой устойчивостью.

3. ОПИСАНИЕ ПРОГРАММНОГО ПАКЕТА

Разработка оформлена как пакет (crate) языка Rust с бинарным приложением и библиотекой. Структура проекта приведена в таблице 1.

Таблица 1 — Структура пакета

Файл	Назначение
Cargo.toml	Манифест пакета: имя, версия, edition, зависимости
src/lib.rs	Корень библиотеки: публичный API, юнит-тесты
src/jacobi.rs	Ядро метода: повороты, обновление A и V, off_sum
src/solver.rs	Решатель JacobiSolver: проверки входа, цикл итераций
src/error.rs	Перечисление JacobiError и его вывод
src/matrix.rs	Генерация случайной симметричной матрицы, вывод
src/console_io.rs	Консольный ввод/вывод и разбор чисел
src/main.rs	Точка входа консольного приложения
tests/solve.rs	Интеграционные тесты
docs/	Сгенерированная документация (cargo doc)
prompts.md	Журнал промтов ассистенту

Публичный интерфейс библиотеки: тип JacobiSolver с методами new(), with_tolerance(), with_max_iterations() и solve(); перечисление ошибок JacobiError; функция off_sum(). Зависимости: nalgebra v0.35 (матричные операции), approx v0.5 (сравнение с погрешностью в тестах).

Ключевые особенности реализации:

- тип solve() возвращает Result<(DVector<f32>, DMatrix<f32>), JacobiError> — собственные числа и матрицу собственных векторов;
- до вызова метода выполняются три проверки входа: квадратность, конечность элементов (отсутствие NaN/Inf) и симметричность;
- для гарантии остановки ограничено максимальное число итераций, по их исчерпании возвращается ошибка DidNotConverge (не сошлось);
- консольное приложение поддерживает три источника матрицы: ручной ввод, случайная генерация и чтение из файла (csv через «;», txt через пробел; дробная часть разделяется точкой или запятой);
- после решения выводится невязка $\|A \cdot V - V \cdot \mathrm{diag}(\lambda)\|$ в качестве численной проверки.

4. ХОД РАБОТЫ

Работа выполнялась поэтапно. На каждом этапе ассистенту давалось задание, он возвращал теорию и структуру (скелет) кода (название функций, параметры и возвращаемые значения без самого алгоритма), а реализацию проверял компилятор (cargo build, cargo clippy). В конечном итоге итоговый код доведён до рабочего состояния.

4.1. Ядро метода (src/jacobi.rs)

Основные функции ядра. rotation_cos_sin вычисляет cosφ и sinφ по формулам (7)–(8), возвращая None, если внедиагональный элемент уже равен нулю (поворот не нужен):

```
pub(crate) fn rotation_cos_sin(a_pp: f32, a_qq: f32, a_pq: f32)
    -> Option<(f32, f32)>
{
    if a_pq == 0.0 {
        return None;
    }
    let c = (a_qq - a_pp) / (2.0 * a_pq);
    let root = (c * c + 1.0).sqrt();
    let t = if c > 0.0 { 1.0 / (c + root) }
            else { 1.0 / (c - root) };
    let cos = 1.0 / (1.0 + t * t).sqrt();
    let sin = t * cos;
    Some((cos, sin))
}
```

Функция rotate_a выполняет обновление матрицы A по формулам (9)–(10) и возвращает квадрат обнуляемого элемента, используемый для поправки нормы S. Функция rotate_v обновляет матрицу собственных векторов V накопленным преобразованием. Функция off_sum вычисляет начальную сумму квадратов внедиагональных элементов S_0.

4.2. Функция входа в решение (src/solver.rs)

Метод solve() выполняет проверки входа, инициализирует V = I, вычисляет S_0 и запускает цикл по лексикографическому обходу пар:

```
for _ in 0..self.max_iterations {
    for p in 0..n - 1 {
        for q in p + 1..n {
            let a_pp = work[(p, p)];
            let a_qq = work[(q, q)];
            let a_pq = work[(p, q)];
            if let Some((cos, sin)) =
                rotation_cos_sin(a_pp, a_qq, a_pq)
            {
                let tg = sin / cos;
                s -= rotate_a(&mut work, p, q, tg, cos, sin);
```

```
                    rotate_v(&mut v, p, q, cos, sin);
                }
            }
        }
        if s.max(0.0) < self.tolerance {
            return Ok((work.diagonal(), v));
        }
    }
```

Ограничение self.max_iterations гарантирует, что цикл не может выполняться бесконечно, даже при вырожденных или слабо обусловленных матрицах. По исчерпании лимита возвращается ошибка DidNotConverge с указанием числа итераций и остаточной нормы.

4.3. Консольный ввод/вывод (src/console_io.rs)

Приложение предлагает выбрать источник матрицы (таблица 2). Разбор чисел унифицирован: принимается точка и запятая в качестве разделителя дробной части, в csv поля разделяются точкой с запятой, в txt и в консоли — пробелом.

Таблица 2 — Источники матрицы

Пункт	Источник	Формат
1	Ручной ввод в консоле	Последовательное введение строк, элементы через пробел
2	Случайная симметричная	Задаётся размерность n, генерируется $A + A^T$
3	Чтение из файла	csv через «;» или txt через пробел; дробная часть . или ,

Дополнительно запрашиваются требуемая точность (по умолчанию 1e-6) и максимальное число итераций (по умолчанию 1000), которые передаются в решатель через билдер-методы with_tolerance() и with_max_iterations().

4.4. Документация и тесты

Документация оформлена док-комментариями //! и /// в исходных файлах и развернута командой cargo doc --no-deps в папку docs/ (вход — docs/jacobi/index.html).

Тесты расположены в секции #[cfg(test)] модуля src/lib.rs (юнит-тесты) и в tests/solve.rs (интеграционные). Покрываются:

- вычисление off_sum для матриц 1×1 и 2×2;

- проверка cosφ и sinφ на эталонном примере из методички;
- проверка обнуления A_pq по условию уравнения (2);
- проверка обновления матрицы A на 5×5 примере с сохранением симметрии;
- инвариант спектрального разложения $A \cdot V \approx V \cdot \mathrm{diag}(\lambda)$ и ортогональность V.

5. РЕЗУЛЬТАТЫ И ОБСУЖДЕНИЕ

Разработка собрана без ошибок (cargo build) и без предупреждений линтера (cargo clippy). Тесты проходят (cargo test).

Пример работы приложения для матрицы 3×3 приведён в таблице 3.

Таблица 3 — Пример результатов

Показатель	**Значение**
Размерность	3 × 3
Собственные числа	$\lambda_1 \approx 0.37$, $\lambda_2 \approx 1.53$, $\lambda_3 \approx 2.41$
Невязка $\|A \cdot V - V \cdot \mathrm{diag}(\lambda)\|$	≈ 1e-6
Сходимость	Достигнута за несколько итераций

Сумма собственных чисел совпадает со следом исходной матрицы, матрица V ортогональна, невязка находится на уровне заданной точности. Это подтверждает корректность реализации: обнуление внедиагональных элементов, обновление V и критерий остановки работают согласованно.

Отдельно проверено поведение на краевых случаях: матрицы 1×1 (вырожденный случай, возвращается единичная V), уже диагональные матрицы ($S_0 = 0$, решение сразу), несимметричная матрица (возвращается ошибка NotSymmetric), матрица с NaN/Inf (ошибка NonFiniteElements).

ЗАКЛЮЧЕНИЕ

В ходе работы разработан пакет на языке Rust, реализующий метод Якоби вычисления собственных чисел и собственных векторов действительной симметричной матрицы.

Выполнены следующие пункты плана:

- реализовано ядро метода: вычисление угла поворота, обновление матрицы A и накопление матрицы собственных векторов V;
- соблюдены требования задания: корректность для любой симметричной матрицы, гарантия остановки (ограничение числа итераций), отсутствие зацикливания;
- разработан публичный интерфейс библиотеки (JacobiSolver, JacobiError), обработка ошибок через Result без паник на некорректных данных;
- создано консольное приложение с тремя источниками ввода матрицы и численной проверкой результата;
- написана документация (cargo doc), юнит- и интеграционные тесты, код проходит cargo clippy без предупреждений.

Практическая ценность работы: полученный пакет можно использовать как библиотека в других проектах Rust, а консольное приложение — как самостоятельный инструмент расчёта.

ПРИЛОЖЕНИЕ А

Исходный код

Полный исходный код доступен по ссылке: https://github.com/Holnstein/mii_2026/tree/main/lab1%20jacobi/src

Основные файлы приведены ниже.

Листинг 1. Файл src/lib.rs

```
pub mod matrix;
pub mod console_io;
pub mod jacobi;
pub mod error;
pub mod solver;

pub use error::JacobiError;
pub use jacobi::off_sum;
pub use solver::JacobiSolver;

#[cfg(test)]
mod tests {
    use approx::relative_eq;
    use nalgebra::DMatrix;
    use crate::jacobi::{rotate_a, rotation_cos_sin};
    use super::*;

    #[test]
    fn off_sum_of_2x2() {
        let a = DMatrix::from_row_slice(2, 2, &[1.0, 3.0, 3.0, 5.0]);
        assert!((off_sum(&a) - 9.0).abs() < 1e-6);
    }

    #[test]
    fn off_sum_of_1x1_is_zero() {
        let a = DMatrix::from_row_slice(1, 1, &[42.0]);
        assert_eq!(off_sum(&a), 0.0);
    }

    #[test]
    fn rotation_check_cos_and_sin_example_from_lab1() {
        let (cos, sin) = rotation_cos_sin(7.0, 19.0, 13.0).unwrap();
        assert!((cos - 0.842).abs() < 1e-2);
        assert!((sin - 0.539).abs() < 1e-2);
    }

    #[test]
    /// Проверка формулы (7): при выборе угла по (9)-(11)
    fn rotation_check_seven_equation_from_lab1() {
        let cases = [
            (7.0, 19.0, 13.0),              // пример из методички, C > 0
            (19.0, 7.0, 13.0),              // C < 0
            (7.0, 19.0, -13.0),             // a_pq < 0
            (100.0, -100.0, 50.0),          // C = 0
```

```
        ];

        for (a_pp, a_qq, a_pq) in cases {
            let (cos, sin) = rotation_cos_sin(a_pp, a_qq, a_pq)
                .expect("a_pq != 0, должен быть Some");

            // формула (6): A_pq' = (cos^2 - sin^2)·a_pq + sin * cos * (a_pp -
a_qq)
            let new_a_pq = (cos * cos - sin * sin) * a_pq
                + sin * cos * (a_pp - a_qq);

            let eps = 1e-5;

            assert!(
                new_a_pq.abs() < eps,
                "A_pq' = {new_a_pq}, ожидалось около 0 с точностью {eps} для \
                 a_pp = {a_pp}, a_qq = {a_qq}, a_pq = {a_pq}"
            );
        }
    }

    #[test]
    // проверка поворота матрицы A, будет падать на маленьких eps из-за
сравнения матриц
    // по методичке
    fn rotate_a_example_lab1() {
        let mut a = DMatrix::from_row_slice(5, 5, &[
            1.0, 4.0, 7.0, 10.0, 13.0,
            4.0, 7.0, 10.0, 13.0, 16.0,
            7.0, 10.0, 13.0, 16.0, 19.0,
            10.0, 13.0, 16.0, 19.0, 22.0,
            13.0, 16.0, 19.0, 22.0, 25.0,
        ]);

        let (cos, sin) = rotation_cos_sin(a[(1, 1)], a[(3, 3)], a[(1,
3)]).unwrap();
        let tg = sin / cos;

        rotate_a(&mut a, 1, 3, tg, cos, sin);

        assert_eq!(a[(1, 3)], 0.0, "A[1][3] должен быть 0");
        assert_eq!(a[(3, 1)], 0.0, "A[4][2] должен быть 0");

        let a_itog = DMatrix::from_row_slice(5, 5, &[
            1.0, -2.02, 7.0, 10.58, 13.0,
            -2.02, -1.32, -0.2, 0.0, 1.62,
            7.0, -0.2, 13.0, 18.86, 19.0,
            10.58, 0.0, 18.86, 27.32, 27.15,
            13.0, 1.62, 19.0, 27.15, 25.0,
        ]);

        for r in 0..a.nrows() {
            for c in 0..a.ncols() {
                assert!(
                    (a[(r, c)] - a[(c, r)]).abs() < 1e-5,
                    "симметрия нарушена в [{r}][{c}]: {} vs {}",
                    a[(r, c)], a[(c, r)]
```

```
                );
            }
        }

        let is_equal = relative_eq!(a, a_itog, epsilon = 1e-2);
        assert!(is_equal, "a={a} a itog = {a_itog}");
    }

}
```

Листинг 2. Файл src/jacobi.rs

```
//! Методы алгоритма Якоби

use nalgebra::DMatrix;

/// Вычисляет косинус и синус угла поворота Гивенса
///
/// Угол подбирается такой, чтобы при применении соответствующего поворота
/// к матрице A элемент Apq занулился
///
/// # Аргументы
///
/// * `a_pp` - диагональный элемент App
/// * `a_qq` - диагональный элемент Aqq
/// * `a_pq` - внедиагональный элемент для зануления Apq
///
/// # Возвращаемые значения
///
/// * `Some((cos, sin))` - если Apq не нуль. Гарантируется `cos > 0`
///   и `|fi| < pi/4` для взятия меньшего корня для тангенса для лучшей
сходимости
/// * `None` - если `Apq == 0` => поворот не нужен
///
pub(crate) fn rotation_cos_sin(a_pp: f32, a_qq: f32, a_pq: f32) ->
Option<(f32, f32)> {
    if a_pq == 0.0 {
        return None;
    }
    let c = (a_qq - a_pp) / (2.0 * a_pq);
    let root = (c * c + 1.0).sqrt();
    let t = if c > 0.0 { 1.0 / (c + root) } else { 1.0 / (c - root) };
    let cos = 1.0 / (1.0 + t * t).sqrt();
    let sin = t * cos;
    Some((cos, sin))
}

/// Выполняется шаг Якоби: зануление Apq и обновление связанных элементов
///
/// Так как матрица A симметрична, то все зеркальные элементы обновляются тоже
(ij = ji)
///
/// # Аргументы
///
/// * `a` - матрица, которая изменяется в методе
/// * `p`, `q` - индексы зануляемого недиагонального элемента
```

```
/// * `t` - тангенс угла поворота, `t == sin / cos`,
/// * `cos`, `sin` - косинус и синус угла поворота, полученные из
`rotation_cos_sin(...)`
///
/// # Возвращаемое значение
///
/// f32 - Квадрат старого значения Apq для обновления суммы `S`
///
/// # Паника
///
/// Метод паникует, если `p == q` (попытка обновить диагональные элементы),
/// или при выходе за границы матрицы (поведение `DMatrix`)
///
pub(crate) fn rotate_a(
    a: &mut DMatrix<f32>,
    p: usize,
    q: usize,
    t: f32,
    cos: f32,
    sin: f32,
) -> f32 {

    assert_ne!(p, q, "индекс столбца не должен быть равен индексу строки");

    let a_pp: f32 = a[(p, p)];
    let a_qq: f32 = a[(q, q)];
    let a_pq: f32 = a[(p, q)];

    a[(p, p)] = a_pp - a_pq * t;
    a[(q, q)] = a_qq + a_pq * t;

    let fraction = sin / (1.0 + cos);
    for r in 0..a.nrows() {
        if r != q && r != p {
            let a_rp = a[(r, p)];
            a[(r, p)] = a_rp - sin * (a[(r, q)] + fraction * a_rp);
            a[(r, q)] = a[(r, q)] + sin * (a_rp - fraction * a[(r, q)]);
            a[(p, r)] = a[(r, p)];
            a[(q, r)] = a[(r, q)];
        }
    }

    a[(p, q)] = 0.0;
    a[(q, p)] = 0.0;

    a_pq.powi(2)
}

/// Обновляет матрицу собственных векторов `V` после шага Якоби,
/// которая будет являться произведением всех промежуточных матриц поворотов,
/// при этом будут меняться только отдельные элементы матрицы для
эффективности
///
/// # Аргументы
///
/// * `v` - матрица собственных векторов, первоначально является единичной
/// * `p`, `q` - индексы зануляемого недиагонального элемента
```

```
/// * `cos`, `sin` - косинус и синус угла поворота, полученные из
`rotation_cos_sin(...)`
///
pub(crate) fn rotate_v(v: &mut DMatrix<f32>, p: usize, q: usize, cos: f32,
sin: f32) {

    for r in 0..v.nrows() {
        let v_rp = v[(r, p)];
        v[(r, p)] = v_rp * cos - v[(r, q)] * sin;
        v[(r, q)] = v_rp * sin + v[(r, q)] * cos;
    }

}

/// Сумма квадратов недиагональных элементов матрицы A
///
/// Соответствует величине `S`. Считается только по верхнему треугольнику (`q
> p`),
/// так как у нас симметричная матрица
///
/// # Аргументы
///
/// * `a` - симметричная матрица, которая изменяется в методе
///
/// # Возвращаемое значение
///
/// f32 - сумма квадратов недиагональных элементов
///
pub fn off_sum(a: &DMatrix<f32>) -> f32 {
    let mut off_sum = 0.0;
    let n  = a.nrows();

    for p in 0..n - 1 {
        for q in p + 1..n {
            off_sum += a[(p, q)] * a[(p, q)];
        }
    }

    off_sum
}
```

Листинг 3. Файл src/solver.rs

```
//! Методы для решения и найденных собственных чисел и векторов для матрицы в
алгоритме Якоби
use nalgebra::{DMatrix, DVector};

use crate::error::JacobiError;
use crate::jacobi::{off_sum, rotate_a, rotate_v, rotation_cos_sin};

pub struct JacobiSolver {
    pub max_iterations: usize,
    pub tolerance: f32,
}

impl JacobiSolver {
```

```
    /// Создание структуры JacobiSolver с дефолтными параметрами (кол-во
итераций и точность)
    pub fn new() -> Self {
        Self {
            max_iterations: 1000,
            tolerance: 1e-6,
        }
    }

    /// Задание точности алгоритма
    ///
    /// # Аргументы
    ///
    /// * `self` - референс текущего модуля, скрывающее название текущего типа
    /// * `eps` - точность
    ///
    /// # Возвращаемые параметры
    ///
    /// `Self` - JacobiSolver с заданной точностью
    ///
    /// # Паника
    ///
    /// Метод паникует, если точность неположительная или точность не является
конечным числом
    pub fn with_tolerance(mut self, eps: f32) -> Self {
        assert!(eps > 0.0, "eps should be a positive number");
        assert!(eps.is_finite(), "eps should be finite");

        self.tolerance = eps;
        self
    }

    /// Задание максимального кол-ва итераций
    ///
    /// # Аргументы
    ///
    /// * `self` - референс текущего модуля, скрывающее название текущего типа
    /// * `iters` - кол-во итераций
    ///
    /// # Возвращаемые параметры
    ///
    /// `Self` - JacobiSolver с заданной точностью
    ///
    /// # Паника
    ///
    /// Паникует, когда кол-во итераций неположительно
    pub fn with_max_iterations(mut self, iters: usize) -> Self {
        assert!(iters > 0, "iters should be a positive number");

        self.max_iterations = iters;
        self
    }

    /// Точка решения алгоритма Якоби
    ///
    /// Алгоритм прекращается либо по окончанию итераций (не сошлось), либо
сумма квадратов
```

```
    /// недиагональных элементов == 0, то есть все внедиагональные элементы
занулились
    ///
    /// # Аргументы
    ///
    /// * `self` - референс текущего модуля, скрывающее название текущего типа
    /// * `a` - симметричная матрица, с которой работает алгоритм
    ///
    /// # Возвращаемые значения
    ///
    /// * `Ok((work.diagonal()), v)` - при успехе возвращаются собственные
числа и матрицу
    ///   собственных векторов
    ///
    /// * `Err(...)` - алгоритм не сошелся, возвращает ошибку с кол-во
итераций и суммой квадратов
    ///
    /// # Паника
    ///
    /// Метод паникует при выходе из границ цикла
    pub fn solve(&self, a: &DMatrix<f32>) -> Result<(DVector<f32>,
DMatrix<f32>), JacobiError> {
        check_square(a)?;
        check_finite(a)?;
        check_symmetric(a)?;

        let n = a.nrows();

        if n < 2 {
            return Ok((a.diagonal(), DMatrix::identity(n, n)));
        }

        let mut work = a.clone();
        let mut v: DMatrix<f32> = DMatrix::identity(n, n);

        let mut s = off_sum(a);

        if s < self.tolerance {
            return Ok((work.diagonal(), v));
        }

        for _ in 0..self.max_iterations {
            for p in 0..n - 1 {
                for q in p + 1..n {
                    let a_pp = work[(p, p)];
                    let a_qq = work[(q, q)];
                    let a_pq = work[(p, q)];

                    if let Some((cos, sin)) = rotation_cos_sin(a_pp, a_qq,
a_pq) {
                        let tg = sin / cos;
                        s -= rotate_a(&mut work, p, q, tg, cos, sin);
                        rotate_v(&mut v, p, q, cos, sin);
                    }
                }
            }

            if s.max(0.0) < self.tolerance {
```

```
                return Ok((work.diagonal(), v));
            }
        }

        Err(JacobiError::DidNotConverge {
            iterations: self.max_iterations,
            residual: s.max(0.0),
        })
    }
}

impl Default for JacobiSolver {
    fn default() -> Self {
        Self::new()
    }
}

/// Проверяет, является ли матрица квадратной
///
/// # Аргументы
///
/// * `a` - матрица для проверки
///
/// # Возвращаемые значения
///
/// * `Ok(())` - успех, матрица квадратична
///
/// * `Err(JacobiError::NotSquare)` - ошибка, матрица не квадратична
///
fn check_square(a: &DMatrix<f32>) -> Result<(), JacobiError> {
    if !a.is_square() {
        return Err(JacobiError::NotSquare)
    }
    Ok(())
}

/// Проверяет, являются ли все элементы матрицы конечными числами (или же NaN
и тд)
///
/// # Аргументы
///
/// * `a` - матрица для проверки
///
/// # Возвращаемые значения
///
/// * `Ok(())` - успех, все числа конечны
///
/// * `Err(JacobiError::NonFiniteElements)` - ошибка, есть хоть одно
неконечное число
///
fn check_finite(a: &DMatrix<f32>) -> Result<(), JacobiError> {
    if a.iter().all(|&x| x.is_finite()) {
        Ok(())
    } else {
        Err(JacobiError::NonFiniteElements)
    }
}
```

```
/// Проверяет, является ли матрица симметричной, то есть Aij == Aji с
определенной точностью
///
/// # Аргументы
///
/// * `a` - матрица для проверки
///
/// # Возвращаемые значения
///
/// * `Ok(())` - успех, матрица симметрична
///
/// * `Err(JacobiError::NotSymmetric)` - ошибка, симметрии нет
///
fn check_symmetric(a: &DMatrix<f32>) -> Result<(), JacobiError> {
    let eps = 1e-4;
    for i in 0..a.nrows() {
        for j in i + 1..a.ncols() {
            let diff = (a[(i, j)] - a[(j, i)]).abs();
            if diff > eps {
                return Err(JacobiError::NotSymmetric)
            }
        }
    }
    Ok(())
}
```

Листинг 4. Файл src/error.rs

```
//! Файл с ошибками для алгоритма Якоби
use std::error::Error;
use std::fmt;

#[derive(Debug, Clone, PartialEq)]
pub enum JacobiError {
    NotSquare,
    NotSymmetric,
    NonFiniteElements,
    DidNotConverge { iterations: usize, residual: f32 },
    EmptyMatrix,
}

impl fmt::Display for JacobiError {
    fn fmt(&self, f: &mut fmt::Formatter<'_>) -> fmt::Result {
        match self {
            Self::NotSquare => write!(f, "матрица не квадратная"),
            Self::NotSymmetric => write!(f, "матрица не симметрична"),
            Self::NonFiniteElements => write!(f, "матрица содержит NaN или Inf
значения"),
            Self::DidNotConverge {iterations, residual} => write!(
                f, "не сошлось за {iterations} итераций (res: {residual})"
            ),
            Self::EmptyMatrix => write!(f, "матрица пустая"),
        }
    }
}

impl Error for JacobiError {}
```

Листинг 5. Файл src/matrix.rs

```
use nalgebra::DMatrix;
#[derive(Default, Debug)]
pub struct MatrixConfig {
    pub dims: usize,
}

/// Создание случайной симметричной матрицы путем сложения случайной матрицы с
ее транспонированной
///
/// # Аргументы
///
/// * `matrix` - конфиг матрицы: порядок матрицы
///
/// # Возвращаемое значение
///
/// * `Dmatrix<f32>` - симметричная матрица с десятичными значениями
///
pub fn create_symmetric_matrix(matrix: MatrixConfig) -> DMatrix<f32> {
    let mtrx: DMatrix<f32> = DMatrix::new_random(matrix.dims, matrix.dims);
    let transposed = mtrx.transpose();

    mtrx + transposed
}

/// Выводит в консоль матрицу
pub fn print_matrix(mtrx: &DMatrix<f32>) {
    println!("mtrx = {:.2}", &mtrx);
}
```

Листинг 6. Файл src/console_io.rs

```
//! Работа консольного приложения
use std::fs;
use std::io::{self, Write};

use nalgebra::{DMatrix, DVector};

/// Выбор для юзера, какой способ чтения матрицы будет использоваться
pub fn read_source_choice() -> Result<u8, io::Error> {
    println!("Источник матрицы:");
    println!("  1 — вручную в консоле (элементы через пробел, по строке на
вход)");
    println!("  2 — случайная симметричная (задать размерность)");
    println!("  3 — из файла (csv через ';', txt через пробел; дробная часть
'.' или ',')");
    print!("Выбор: ");
    io::stdout().flush()?;

    let mut line = String::new();
    io::stdin().read_line(&mut line)?;
```

```
    line.trim().parse::<u8>().map_err(|err| {
        io::Error::new(
            io::ErrorKind::InvalidInput,
            format!("'{line}' — не пункт меню: {err}"),
        )
    })
}

/// Чтение размерности матрицы
pub fn read_dims() -> Result<usize, io::Error> {
    print!("Введите размерность матрицы (n): ");
    io::stdout().flush()?;

    let mut line = String::new();
    io::stdin().read_line(&mut line)?;

    line.trim().parse::<usize>().map_err(|err| {
        io::Error::new(
            io::ErrorKind::InvalidInput,
            format!("'{line}' is not a number: {err}"),
        )
    })
}

/// Чтение точности (эпсилон) с консоли
pub fn read_tolerance() -> Result<f32, io::Error> {
    print!("Введите точность (эпсилон, по умолчанию 1e-6): ");
    io::stdout().flush()?;

    let mut line = String::new();
    io::stdin().read_line(&mut line)?;

    let trimmed = line.trim();
    if trimmed.is_empty() {
        return Ok(1e-6);
    }

    let eps: f32 = trimmed.replace(',', ".").parse().map_err(|err| {
        io::Error::new(
            io::ErrorKind::InvalidInput,
            format!("'{trimmed}' - не число: {err}"),
        )
    })?;

    if !(eps > 0.0 && eps.is_finite()) {
        return Err(io::Error::new(
            io::ErrorKind::InvalidInput,
            "точность должна быть положительным конечным числом",
        ));
    }

    Ok(eps)
}

/// Чтение максимального количества итераций с консоли
pub fn read_max_iterations() -> Result<usize, io::Error> {
    print!("Введите макс. кол-во итераций (по умолчанию 1000): ");
```

```
    io::stdout().flush()?;

    let mut line = String::new();
    io::stdin().read_line(&mut line)?;

    let trimmed = line.trim();
    if trimmed.is_empty() {
        return Ok(1000);
    }

    let iters: usize = trimmed.parse().map_err(|err| {
        io::Error::new(
            io::ErrorKind::InvalidInput,
            format!("'{trimmed}' — не целое число: {err}"),
        )
    })?;

    if iters == 0 {
        return Err(io::Error::new(
            io::ErrorKind::InvalidInput,
            "количество итераций должно быть больше 0",
        ));
    }

    Ok(iters)
}

/// Чтение полного пути до матрицы, если выбрано чтение из файла
pub fn read_path() -> Result<String, io::Error> {
    print!("Путь к файлу: ");
    io::stdout().flush()?;

    let mut line = String::new();
    io::stdin().read_line(&mut line)?;

    let path = line.trim();
    if path.is_empty() {
        Err(io::Error::new(io::ErrorKind::InvalidInput, "пустой путь"))
    } else {
        Ok(path.to_string())
    }
}

/// Чтение матрицы с консольного ввода
pub fn read_matrix_from_console(n: usize) -> Result<DMatrix<f32>, io::Error> {
    let mut data = Vec::with_capacity(n * n);

    for row in 0..n {
        println!("Строка {} ({} числа через пробел):", row + 1, n);
        io::stdout().flush()?;

        let mut line = String::new();
        io::stdin().read_line(&mut line)?;

        let numbers = parse_numbers(&line)?;
        if numbers.len() != n {
            return Err(io::Error::new(
```

```
                io::ErrorKind::InvalidInput,
                format!(
                    "строка {}: ожидалось {n} чисел, получено {}",
                    row + 1,
                    numbers.len()
                ),
            ));
        }
        data.extend(numbers);
    }

    Ok(DMatrix::from_row_slice(n, n, &data))
}

/// Чтение матрицы из файла
pub fn read_matrix_from_file(path: &str) -> Result<DMatrix<f32>, io::Error> {
    let content = fs::read_to_string(path)?;

    let mut data = Vec::new();
    let mut n = 0usize;

    for (idx, line) in content.lines().enumerate() {
        let line = line.trim();
        if line.is_empty() {
            continue;
        }

        let numbers = parse_numbers(line)?;
        if n == 0 {
            n = numbers.len();
        } else if numbers.len() != n {
            return Err(io::Error::new(
                io::ErrorKind::InvalidData,
                format!(
                    "строка {}: ожидалось {n} чисел, получено {}",
                    idx + 1,
                    numbers.len()
                ),
            ));
        }
        data.extend(numbers);
    }

    if n == 0 || data.len() != n * n {
        return Err(io::Error::new(
            io::ErrorKind::InvalidData,
            format!(
                "нужна квадратная матрица: строк {n}, всего чисел {}",
                data.len()
            ),
        ));
    }

    Ok(DMatrix::from_row_slice(n, n, &data))
}

/// Вывод решения
///
```

```
/// # Аргументы
///
/// * `eigenvalues` - собственные числа матрицы (их сумма равна сумме
диагональных элементов)
/// * `eigenvectors` - собственные векторы, представленные столбцами в одной
матрице (`V`)
pub fn print_solution(eigenvalues: &DVector<f32>, eigenvectors: &DMatrix<f32>)
{
    println!("Собственные числа:");
    for (i, &lambda) in eigenvalues.iter().enumerate() {
        println!("  lambda[{i}] = {lambda:.6}");
    }

    println!("Собственные векторы (столбцы):");
    println!("{eigenvectors:.4}");
}

/// Вывод невязки в консоль из формуды A * V = V * lambda
pub fn print_residual(
    a: &DMatrix<f32>,
    eigenvalues: &DVector<f32>,
    eigenvectors: &DMatrix<f32>,
) {
    let lambda_diag = DMatrix::from_diagonal(eigenvalues);
    let residual = (a * eigenvectors - eigenvectors * lambda_diag).norm();
    println!("Невязка ||A*V - V*diag(lambda)|| = {residual:.6e}");
}

/// Парсинг чисел с консоли или файла
///
/// # Аргументы
///
/// * `line` - строка с текущей строкой читаемой матрицы
///
/// # Возвращаемые значения
///
/// Result<Vec<f32>> - вектор значений для матрицы
///
/// io::Error - ошибка ввода
///
fn parse_numbers(line: &str) -> Result<Vec<f32>, io::Error> {
    line.split(|c: char| c == ';' || c.is_whitespace())
        .filter(|token| !token.is_empty())
        .map(|token| {
            token.replace(',', ".").parse::<f32>().map_err(|err| {
                io::Error::new(
                    io::ErrorKind::InvalidInput,
                    format!("'{token}' — не число: {err}"),
                )
            })
        })
        .collect()
}
```

Листинг 7. Файл src/main.rs

```
//! Основной файл запуска алгоритма метода Якоби для поиска собственных чисел
и векторов
//! симметричной матрицы
use std::io;
use std::io::Write;
use nalgebra::DMatrix;

use jacobi::console_io;
use jacobi::matrix;
use jacobi::matrix::MatrixConfig;
use jacobi::{off_sum, JacobiSolver};


/// Чтение матрицы с консольного инпута
fn read_input_matrix() -> Result<DMatrix<f32>, io::Error> {
    match console_io::read_source_choice()? {
        1 => {
            let dims = console_io::read_dims()?;
            if dims == 0 {
                return Err(io::Error::new(
                    io::ErrorKind::InvalidInput,
                    "размерность должна быть не меньше 1",
                ));
            }
            console_io::read_matrix_from_console(dims)
        }
        2 => {
            let dims = console_io::read_dims()?;
            if dims == 0 {
                return Err(io::Error::new(
                    io::ErrorKind::InvalidInput,
                    "размерность должна быть не меньше 1",
                ));
            }
            Ok(matrix::create_symmetric_matrix(MatrixConfig { dims }))
        }
        3 => {
            let path = console_io::read_path()?;
            console_io::read_matrix_from_file(&path)
        }
        other => Err(io::Error::new(
            io::ErrorKind::InvalidInput,
            format!("пункт меню должен быть 1, 2 или 3, получено {other}"),
        )),
    }
}

/// Главный двигатель процесса
///
/// # Examples
/// ```
/// let a = 1;
/// assert!(a, 1);
/// ```
///
fn main() {
    let mtrx = match read_input_matrix() {
        Ok(m) => m,
        Err(e) => {
```

```
            eprintln!("Ошибка ввода: {e}");
            return;
        }
    };

    let (rows, cols) = (mtrx.nrows(), mtrx.ncols());
    println!("Исходная матрица A ({rows}x{cols}):");
    matrix::print_matrix(&mtrx);

    let s0 = off_sum(&mtrx);
    println!("S0 (сумма квадратов недиагональных элементов) = {s0:.6e}");

    let tolerance = match console_io::read_tolerance() {
        Ok(t) => t,
        Err(e) => {
            eprintln!("Ошибка ввода точности: {e}");
            return;
        }
    };

    let max_iterations = match console_io::read_max_iterations() {
        Ok(m) => m,
        Err(e) => {
            eprintln!("Ошибка ввода итераций: {e}");
            return;
        }
    };

    println!("Параметры: точность = {tolerance}, макс. итераций =
{max_iterations}");

    let solver = JacobiSolver::new()
        .with_tolerance(tolerance)
        .with_max_iterations(max_iterations);

    match solver.solve(&mtrx) {
        Ok((eigenvalues, eigenvectors)) => {
            console_io::print_solution(&eigenvalues, &eigenvectors);
            console_io::print_residual(&mtrx, &eigenvalues, &eigenvectors);
        }
        Err(e) => {
            eprintln!("Ошибка решения: {e}");
        }
    }

    println!("Нажмите Enter для выхода...");
    io::stdout().flush().unwrap();
    let mut input = String::new();
    io::stdin().read_line(&mut input).unwrap();
}
```

ПРИЛОЖЕНИЕ Б

Журнал промтов

Полный журнал промтов доступен по ссылке: https://github.com/Holnstein/mii_2026/blob/main/lab1%20jacobi/prompts.md

Сокращённая выдержка (по датам):

Дата	Содержание запроса
19.09	Создание AGENTS.md, изучение методички и постановка задачи
20.09	Теория метода: спектральное разложение, формулы 7–19
21.09	Разбор вывода формулы (7) и квадратного уравнения на tg φ
22.09	Работа с DMatrix<f32> в nalgebra, сравнение библиотек
23.09	Паттерн Result и перечисление ошибок JacobiError
02.10	Устойчивая ветвь tan φ, замер точности, каркас rotate_a
03.10	Обновление V, объяснение инкрементального накопления
12.10	Псевдоалгоритм solve, лексикографический обход, анализ бесконечных циклов
15.10	Консольное приложение: console_io и main
22.10	Публичный интерфейс, интеграционный файл тестов, проверка программы
22.10	Три источника ввода: консоль, случайная генерация, файл csv/txt
29.10	Точность и число итераций с консоли; сборка документации cargo doc
29.10	Уточнение пункта ТЗ о превращении программы в пакет

ИСТОЧНИКИ

1. Репозиторий проекта: https://github.com/Holnstein/mii_2026

2. Исходный код (src): https://github.com/Holnstein/mii_2026/tree/main/lab1%20jacobi/src

3. Журнал промтов: https://github.com/Holnstein/mii_2026/blob/main/lab1%20jacobi/prompts.md

4. Задание от преподавателя: https://github.com/Holnstein/mii_2026/blob/main/lab1%20jacobi/lab1.pdf

5. Вход на страницу документации:

https://github.com/Holnstein/mii_2026/blob/main/lab1%20jacobi/docs/jacobi/index.html

6. Сам отчет доступен по ссылке:

https://github.com/Holnstein/mii_2026/blob/main/lab1%20jacobi/report/Лабораторная_работа_1_Метод_Якоби.docx